

Intelligent Robotics

Consolidated Study Notes

Foundations, autonomy and robot architectures

Key idea. A robot connects perception to action in the physical world. The central design question is how to organize that connection when the world cannot be fully specified in advance.

Current coverage: supplied slide sets 02, 03 and 04, integrated with the corresponding passages of Robin R. Murphy's *Introduction to AI Robotics* (2000).

These notes condense repeated slides and illustrative catalogues into definitions, comparisons and worked examples. They retain the English terminology of the course. They are an original synthesis, rather than a reproduction of the source material.

How to study. Read Sections 1–5 in order, then answer the questions in Section 6 without looking at the answers. Use the reading anchors to return to a specific source only when a concept remains unclear.

Scope. The supplied book file contains only its first 124 PDF pages. Later chapters listed in its table of contents are not available here. Slide sets 03 and 04 retain A.Y. 2024–25 cover labels; their filenames identify the supplied lessons. The book supplement separates available Chapter 3 foundations from the introductory slide content. Later exam topics are tracked separately in the repository handoff. Product catalogues are condensed. Version 0.3 adds the slide-specific terminology and examples (tool/agent, the four S04 keyword pairs, Watt's governor, classical AI versus intelligent robotics, slide architecture examples) and Murphy's primitive table, STRIPS difference table and hierarchical evaluation.

Contents

1	What makes a robot intelligent?	3
1.1	Robot, robotics and intelligent robot	3
1.2	How robotics developed	3
1.3	The fundamental elements	4
1.4	A taxonomy with independent dimensions	4
1.5	Sense–Plan–Act	4
1.6	Classical AI paradigm versus intelligent robotics paradigm	5
1.7	Where artificial intelligence enters	5
1.8	Teleoperation, telepresence and supervisory control	6
2	From industrial tools to collaborating agents	7
2.1	Two historical pressures	7
2.2	Murphy’s timeline: from tools to agents	7
2.3	Industrial revolutions: the organizing idea	8
2.4	Collaborative robotics and shared intelligence	8
2.5	What the illustrations are meant to show	9
3	Automation, autonomy and assumptions about the world	9
3.1	What “autonomy” means	9
3.2	Automation versus autonomy	10
3.3	Closed World Assumption	10
3.4	Open World Assumption	11
3.5	Do not confuse closed world with closed-loop control	11
3.6	Greenfield and brownfield automation	11
3.7	A toy planning problem and its hidden assumptions	12
3.8	The frame problem and world-model maintenance	13
3.9	Choosing how much autonomy to use	13
4	Organizing intelligence: robot paradigms and architectures	14
4.1	Paradigm versus architecture	14
4.2	The Hierarchical Paradigm	14
4.3	The Reactive Paradigm	16
4.4	Perception for action and perception for recognition	16
4.5	The Hybrid Deliberative/Reactive Paradigm	17
4.6	Why the interface is difficult	17
4.7	The three paradigms compared	18
4.8	Evaluating an architecture	18
5	Book supplement: biological foundations of behavior	19
5.1	Biological inspiration and computational theory	19
5.2	Behavior categories and acquisition	19
5.3	Innate Releasing Mechanisms and coordination	19
5.4	Action–perception cycle and two uses of perception	20
5.5	Schema theory: connecting perception and motion	20
5.6	Transfer to robots: what remains difficult	20
6	Revision: explain, compare, apply	21
6.1	The essential chain of reasoning	21
6.2	Questions with concise model answers	21
6.3	One scenario across the whole course	23
6.4	Common mistakes to avoid	24
7	Source map and editorial scope	24
7.1	Source identifiers	24
7.2	Which material supports each part?	25
7.3	Editorial choices	25

1 What makes a robot intelligent?

Reading anchors: S02, PDF pp. 2–32; Murphy, printed pp. 2–10, 13–16, 21–24 and 28–37.

1.1 Robot, robotics and intelligent robot

Robotics studies machines that replace or assist humans in physical tasks and decision making. Its organizing idea is the *intelligent connection of perception to action*: sensing is useful because it supports an action, and actions change what will be sensed next.

A **robot** is a programmable physical mechanism with actuation and some capacity to execute intended tasks. The definition attributed to ISO 8373:2012 in the slides emphasizes an actuated mechanism, programming in two or more axes, a degree of autonomy, and movement within its environment: “an actuated mechanism programmable in two or more axes with a certain degree of autonomy, moving within its environment, to perform intended tasks.” S02 adds an informal reading: a robot is an object able to move in space, with at least some actuators and some sensors, and capable of being programmed in some way.

An **intelligent robot** uses sensory information to select or modify its actions in response to its environment. A machine can be a robot while mainly repeating a preprogrammed sequence. Intelligence becomes especially useful when the relevant situation cannot be completely predicted beforehand.

Murphy’s working definition emphasizes a *mechanical creature which can function autonomously*: it should adapt to reasonable changes in its environment or itself while pursuing its goal. This explains the emphasis on adaptation in AI robotics.

The word *robot* comes from the Czech *robota* (hard work): Karel Čapek used it for the artificial workers replacing humans in his play *R.U.R.* (1920). The etymology explains the association with work; Murphy adds that appearance is not a useful definition either, since robots take whatever shape gets the job done (a vacuum-cleaning robot looks like a vacuum cleaner, not a janitor).

1.2 How robotics developed

S02 summarizes the development as a sequence of shifts:

- **Mid-20th century:** research begins to link human intelligence and machines.
- **1960s:** early robots emerge from **numerical control** and **teleoperation** technologies; integrated circuits and digital computers make robots programmable.
- **Late 1970s:** industrial robots automate **flexible manufacturing systems**.
- **1980s:** robotics is defined as the science studying the *intelligent connection of perception to action*.
- **1990s:** robots replace humans in challenging environments (**field robotics**) or improve quality of life (**service robotics**).

Key idea. Physical embodiment matters: a software agent may perceive and decide, but a robot must also deal with motion, contact, energy and the consequences of acting in the physical world.

1.3 The fundamental elements

Element	Function	Typical examples	Disciplines (S02 p. 9)
Sensors	Measure the robot or its environment.	Camera, range sensor, encoder, force sensor.	Electronics, bioengineering.
Information processing	Convert measurements into useful information and decisions.	Perception, localization, planning, control.	Computer science, electronics, telecommunication.
Actuators	Turn commands into physical action.	Wheel motor, manipulator joint, gripper.	Automation, electronics, mechanics.

Computer engineering contributes real-time operating systems, programming languages, data-analysis algorithms, AI, machine learning and perception processing. The mechanical structure determines the motions and interactions that are physically possible. Good planning cannot compensate for an actuator that cannot perform the requested motion; accurate motors cannot compensate for an incorrect interpretation of the scene.

A **degree of freedom (DOF)** is an independent motion variable. A manipulator's joint motions determine its possible configurations; redundant degrees of freedom can provide different joint arrangements for the same task motion. **Kinematics** concerns geometrical motion relationships; **dynamics** concerns motion together with forces, masses and mechanical effects.

A **teach pendant** lets an operator guide and record robot motions for later execution. A traditional **Automatic Guided Vehicle (AGV)** follows a designed route, such as a wire or marked trail. Stopping when that route is obstructed and autonomously finding another route are different capabilities.

1.4 A taxonomy with independent dimensions

The slide taxonomy should be read along several axes, rather than as a list of mutually exclusive robot types:

- **Physical capability:** mobile robots change their location; manipulation robots act on objects. A mobile manipulator does both.
- **Operating domain:** ground, marine, aerial or space.
- **Application:** industrial, agricultural, medical, field or service robotics.
- **Control allocation:** teleoperated, semi-autonomous or autonomous for a specified task.

For example, a rover can be a mobile space robot with autonomous obstacle avoidance and human-supervised mission goals. A robotic surgical system can be a medical manipulator operated by a human. The application or shape alone does not establish autonomy.

Advanced robotics studies robots with high autonomy in unstructured environments. **Field robots** operate where humans cannot safely work (e.g. a planetary rover, a humanoid in a disaster-response challenge). **Service robots** operate in human environments to improve quality of life (e.g. a vacuum-cleaning robot). Both require rich sensing, robust decision making and autonomy, because their surroundings are less structured than an engineered production cell.

1.5 Sense–Plan–Act

The three primitives describe functions, not necessarily three separate hardware devices:

1. **SENSE:** acquire measurements and extract information. Signal processing, computer vision and perception contribute here.
2. **PLAN:** choose actions to achieve a goal using available information. Localization and mapping provide information needed by navigation and decision making; their exact placement depends on the architecture.
3. **ACT:** execute the chosen action through motion control or manipulation.



The action changes the environment and the robot's state, so new observations close the loop. S02 calls this *many robots, one loop*: rovers, cars, manipulators and service robots all fit it. The diagram introduces functions; it does not imply that every robot must wait for a complete planning cycle before reacting.

Murphy defines the primitives by their inputs and outputs (Fig. I.2). This table is the basis for describing each paradigm in Section 4:

Primitive	Input	Output
SENSE	Sensor data.	Sensed information (useful to other functions).
PLAN	Information (sensed and/or cognitive, i.e. knowledge of how the world works).	Directives: tasks such as “go down the hall, turn left”.
ACT	Sensed information or directives.	Actuator commands, e.g. a turning velocity.

Example: delivering a package. SENSE detects a corridor and an obstacle; PLAN selects a route to the delivery room; ACT commands the wheels. When the corridor becomes blocked, the robot must detect the change and adapt its behavior or route. Replaying the original wheel commands is insufficient.

1.6 Classical AI paradigm versus intelligent robotics paradigm

S02 contrasts two pictures (pp. 30–31). Both contain PERCEPTION, REASONING (AI algorithms, decision-making strategies) and ACTION.

- In the **classical artificial intelligence paradigm**, illustrated by chess, reasoning operates on an abstract, fully specified problem: perceive, reason, act as a one-way $\text{SENSE} \Rightarrow \text{PLAN} \Rightarrow \text{ACT}$ chain. The world is a symbolic board whose state is known exactly and changes only through legal moves.
- In the **intelligent robotics paradigm**, the same elements form a *closed loop through the real environment*. Actions change the world, the world is perceived again, and **autonomous behaviour** emerges from this continuous interaction. The world is physical, partially known and changes independently of the robot.

The difference is embodiment and situatedness: intelligence is judged by behavior in the physical world, not by solving an abstract problem. This motivates the open-world assumption and the reactive and hybrid paradigms discussed later.

1.7 Where artificial intelligence enters

Murphy groups AI contributions into seven areas. The useful distinction is the problem each area solves:

Area	Role in a robot
Knowledge representation	Represent the world, the task and the robot's capabilities.
Natural language understanding	Interpret a user's request and its context.
Learning	Improve a model or behavior from experience.
Planning and problem solving	Find actions that achieve a goal and revise failed solutions.
Inference	Derive conclusions from available knowledge or incomplete observations.
Search	Explore alternatives in a problem's search space.
Vision	Extract task-relevant information from images.

AI is broader than machine learning. A symbolic planner or a search algorithm is an AI technique even if it has no learning component. Likewise, *search* may mean exploring possible action sequences rather than physically looking for an object.

1.8 Teleoperation, telepresence and supervisory control

Teleoperation places a human in the control loop at a distance. Murphy presents it as an intermediate solution for tasks that needed automation but for which robots could not yet be adequately programmed. A local workstation provides displays and controls; the remote robot provides sensors, effectors, power and, where needed, mobility. A communication link carries observations and commands.

This is useful when perception, dexterity or unusual decisions benefit from human judgment. Its limitations include restricted field of view, bandwidth, communication delay, cognitive fatigue and the demand for trained operators. A delayed image describes an earlier state; the corresponding command may arrive after the situation has changed.

Murphy lists six conditions favoring teleoperation: the task is unstructured and nonrepetitive; the workspace cannot be engineered for conventional manipulation; some steps need dexterous hand-eye coordination; some need advanced recognition or situational awareness; display requirements fit the communication link; and trained operators are available. A **predictive display** can show a simulated command outcome before the delayed robot response arrives, but its prediction still depends on a suitable model.

Telepresence improves the operator's experience of the remote environment through more natural sensory feedback and interaction. It concerns the interface and the sense of being present remotely; it does not itself give the robot more independent decision-making capability.

Semi-autonomous control, also called **supervisory control** in Murphy, delegates parts of the task:

- **Continuous assistance / shared control:** the operator and robot share control, or the operator monitors a delegated action and can take over. Routine motion may be automated while delicate manipulation remains human-controlled.
- **Control trading:** the human initiates a task that the robot completes autonomously, then gives another command or interrupts it. This reduces continuous communication demands only if the robot can reliably handle the delegated task.

Key idea. Autonomy is task-dependent. A robot may act independently during navigation and still need a human for mission selection or difficult manipulation.

2 From industrial tools to collaborating agents

Reading anchors: S03, PDF pp. 2–27; S02, PDF p. 32; Murphy, printed pp. 13–28.

2.1 Two historical pressures

Murphy summarizes useful applications through the **three D's: dirty, dull and dangerous** work. These motivations include industrial repetition, hazardous handling and humanitarian tasks such as search and rescue. Adoption also involves economic, social and cultural factors. A **Luddite**, in the book's historical discussion, is associated with opposition to technological change; technical capability alone does not establish that people will accept an application.

The early development of robotics followed two related needs. Numerical control and manufacturing pushed toward repeatable, programmable motion. Telemanipulation pushed toward performing physical work while separating the human from a dangerous workspace, particularly in nuclear applications.

Murphy describes a fork between **industrial manipulators** and **AI robotics**. Manufacturing emphasized speed, precision and repeatability in engineered environments. Space exploration emphasized sensing and adaptation, since a rover could encounter terrain or events that had not been specified beforehand. These are different design priorities, rather than a permanent boundary between industrial and intelligent robots.

Milestone	Contribution	Concept to remember
Telemanipulators	Remote handling with human control.	Separate the operator from the physical hazard.
Unimate / Unimation	Programmable industrial manipulation.	Repeat physical work in a structured workspace.
Dartmouth, 1956	The term artificial intelligence, associated with John McCarthy.	Study computation for intelligent behavior.
Shakey, 1966–72	Integrated perception, world modeling and planning at SRI.	A mobile robot reasons about actions toward a goal.
Collaborative robotics	Human and robot work in a shared environment and task.	Allocate responsibilities across both participants.

Unimate (S03 p. 16). In 1956 George Devol and Joe Engelberger founded **Unimation** (Universal Animation), which developed **Unimate**, the first industrial robot, in 1959. It weighed two tons, was controlled by a program stored on a magnetic drum, used hydraulic actuators, was programmed in joint coordinates, and had an accuracy of 1/10,000 of an inch. It is the prototype of a *tool*: precise repetition in an engineered workspace.

Shakey (S03 p. 22). Developed at SRI's Artificial Intelligence Center from 1966 to 1972, Shakey was the first mobile robot to reason about its actions, using programs for perception, world modeling and planning to achieve goals given by a user. It introduced the Hierarchical Paradigm and STRIPS (Section 3). Murphy's illustration gives a narrower interval; the integration of sensing, a world model and planning is the point to remember.

2.2 Murphy's timeline: from tools to agents

S03 (pp. 15, 25) shows Murphy's timeline (2nd ed., 2019) on two tracks. **Industrial robots** (Unimate, SCARA, Kiva/Amazon warehouse systems) are positioned as **tools**; **mobile robots**

(Shakey, Mars rovers, drones, Roomba) move toward **agents**. Each period was driven by an external need:

- WWII and nuclear weapon production: telemanipulation and the nuclear industry.
- The space race: planetary exploration and Mars rovers.
- The Vietnam war: early unmanned aircraft (Firebee, Lightning Bug).
- The Iraq and Afghanistan wars: Predator, weaponized Predator and small unmanned aerial systems.

The final version of the slide adds **collaborative robots (HRI)** as the newest branch. The axis tool → agent anticipates the automation/autonomy distinction of Section 3.

2.3 Industrial revolutions: the organizing idea

1. **Industry 1.0** (first steam loom, c. 1780): mechanization through steam power.
2. **Industry 2.0** (first assembly line, c. 1870): electrification, assembly lines and mass production.
3. **Industry 3.0** (first PLC, c. 1970): electronic control, computers and programmable automation.
4. **Industry 4.0** (today): connected production systems in which machines, data and information technologies are integrated.

The enabling technologies shown in S03 include **Big Data**, the **Internet of Things**, **robotic automation**, **cloud computing and cybersecurity**, **human-machine interfaces**, and **additive manufacturing**. Together they support connected observation, information exchange, production and interaction. Connectivity supplies information; the robot still needs a suitable decision and control architecture to use it.

The dates are approximate teaching landmarks. The useful progression is from mechanizing an operation to automating it and then connecting it to a wider information system.

Industry 5.0, as presented in S03, complements Industry 4.0 by emphasizing a **human-centric**, **sustainable** and **resilient** industry. Productivity is part of the design problem, alongside worker wellbeing, environmental limits and the ability to cope with disruptions.

2.4 Collaborative robotics and shared intelligence

Collaborative robotics is a new paradigm in industrial robotics in which humans and robots share the same environment and collaborate at the same tasks (S03 p. 11). S03 lists three features of **collaborative robots (cobots)**: humans and robots work at the same task; protective fences are not needed; they are particularly interesting for **small and medium-sized enterprises (SMEs)** because of reduced cost and footprint. Cobots were about 10% of the global robot market in 2025, growing by roughly 20% per year. The slide of Rodney Brooks showing a child how to approach a dual-arm manipulator without fear of being hit illustrates the change from fenced cells. The system should combine the robot's repeatability and physical capability with human judgment, flexibility and task knowledge.

The slides ask: *Where is the intelligence located? Should the robot execute commands or interpret them?* The **shared-intelligence approach** answers by distributing useful capabilities across the human and the robot. A command can specify a goal while leaving some execution decisions to the robot.

Example: assisted assembly. The human selects and aligns a component. The robot holds it or executes a repetitive motion. Sensors provide information about contact or task progress, and the

robot adjusts the execution. Designing the combined task requires deciding who chooses the next step, who monitors success, and how an intervention is handled.

Three terms must remain distinct:

- **Collaborative robotics:** a human–robot work arrangement.
- **Shared control:** an allocation of control authority between human and robot.
- **Hybrid deliberative/reactive architecture:** an internal organization combining planning and reactive behaviors.

A collaborative robot can use a hybrid architecture, but the two expressions describe different aspects of the system.

2.5 What the illustrations are meant to show

The contrast between a **dark factory** and a collaborative workspace introduces a design choice: production can aim to remove people from particular operations or to integrate human and robot contributions. Neither diagram alone predicts the effect on employment.

S03 contrasts traditional fenced industrial cells with collaborative applications. The conceptual point is closer interaction. Whether protective separation can be removed depends on the complete application, including the tool, workpiece, motion and protective measures. Treat the slide’s “no fences” wording as an illustration of a possible arrangement, rather than a universal property of every collaborative robot.

Humanoids in factories (S03 p. 27). Interest is high but the market is early. *Why the interest:* progress in AI, sensors and control systems, and the ability to operate in environments designed for humans. *Barriers to scale:* safety standards, training and maintenance costs, manipulation capabilities and economic sustainability. The IFR warns to distinguish potential from commercial maturity: many applications remain specialized and still rely on human teleoperation. Human-like appearance does not establish autonomous competence. The same distinction applies to personal robots and androids shown in S02.

Key idea. Robot form, collaboration and autonomy answer different questions: what the machine is built like, how it works with people, and which decisions it can make for a specified task.

3 Automation, autonomy and assumptions about the world

Reading anchors: S04, PDF pp. 2–18; Murphy, printed pp. 13–16, 42–53.

3.1 What “autonomy” means

The dictionary definition (Merriam-Webster) is *the quality or state of being self-governing, self-directing freedom and especially moral independence*. S04 notes that this reading causes the **myth of evil robots**: it suggests machines with their own moral will.

In robotics, *self-governing* comes instead from the **mechanical tradition**: James Watt controlled his steam engine with a **self-governor** (the centrifugal governor), a device that regulates the machine without a human. Autonomy is therefore about a machine regulating its own behavior toward a goal, not about moral independence.

Bounded rationality adds that all cognitive agents have limits: information, time, memory, computation and available actions. Robotic autonomy means independent operation within a task and an operating scope.

3.2 Automation versus autonomy

The S04 definitions (adapted from Murphy) should be known almost verbatim:

- **Automation** is about physically-situated **tools** performing highly repetitive, pre-planned actions for well-modeled tasks under the **closed world assumption**.
- **Autonomy** is about physically-situated **agents** that not only perform actions but can also adapt to the **open world**, where the environment and tasks are not known a priori, by generating new plans, monitoring and changing plans, and learning within the constraints of their bounded rationality.

Practical examples in S04: an industrial SCARA manipulator (automation) and a Mars rover (autonomy).

Why the difference matters (S04 p. 13). Automation delegates a *small set of repetitious tasks* in a closed world; its focus is on formal, stable control loops. Autonomy delegates a *variety of tasks* in dynamic environments in an open world; its focus is on artificial intelligence.

S04 (pp. 14–18) compares them along four dimensions, each with a keyword pair:

Dimension	Automation	Autonomy
Plans	Execution: plan once, then repeat that plan forever.	Generation: constantly generating new plans.
Actions	Deterministic: the system can be modeled deterministically.	Non-deterministic: the system is too complex to model deterministically.
Models	Closed world: the model contains everything.	Open world: models will only be partial.
Knowledge representation	Signals: control or decision making at the signal level.	Symbols: control or decision making with symbols or labels.

The balance diagrams in S04 emphasize a **continuum**, with different balances for plans, actions, models and knowledge representation. A robot can generate task plans while executing some motions predictably; it can use a partial environment model together with precise signal-level control. Describe the capability being delegated, rather than assigning the whole robot to a single extreme.

The “class discussion” slide asks where a given system sits on each slider. Real robots combine them: an autonomous robot still uses low-level signal control, and an automated machine may contain sensors or symbolic decisions. The answer to “how much autonomy do I need?” is given dimension by dimension.

3.3 Closed World Assumption

Under the **Closed World Assumption (CWA)**, the stored model is treated as complete for the relevant domain. In its logical reading, a fact not represented as true is treated as false. In the engineering reading of S04 p. 7: everything relevant is known a priori and there are no surprises; everything relevant can be completely modeled; if the world is modeled accurately enough, one can create *stable control loops* to respond to all expected situations; and if the world is controlled, sensing can be minimized or eliminated.

Example. A symbolic database lists all obstacles in a work area. Under CWA, an obstacle not listed is treated as absent. This is useful only if the database is sufficiently complete and remains valid for that use.

CWA makes reasoning simpler: a planner can evaluate a fixed set of states and actions. In a

controlled workspace, the designer constrains part positions, timing and disturbances, reducing the external sensing required.

3.4 Open World Assumption

S04 p. 11: under the **Open World Assumption (OWA)**, models may be available but are only partially (and unpredictably) correct, so the robot must be able to sense relevant aspects of the world to adapt its actions dynamically, i.e. act as an agent. The open world “is associated with being autonomous”. Logically, absence of knowledge is not proof of falsity. An unrecorded fact may be true or false: it is unknown from the available information. In robotics, operating in an open world means expecting relevant unknowns, incomplete models and changes that cannot all be enumerated in advance.

Example. A corridor has no obstacle recorded in the map. This does not establish that it is currently clear: a person may have entered since the map was created. The robot must use current observations and update its decisions.

An open world does not mean a robot has no model. It means that the model is partial and must be treated accordingly. Sensing, execution monitoring and adaptation connect the model to the actual situation.

Key idea. CWA: “not known true” is treated as false. OWA: “not known true” remains unknown. For a robot, an old map is not sufficient evidence that a route is clear now.

3.5 Do not confuse closed world with closed-loop control

Closed-loop control uses feedback to compare a desired value with a measured value and correct the action. For example,

$$e(t) = r(t) - y(t), \quad u(t) = K_p e(t),$$

where $r(t)$ is a desired position, $y(t)$ a measured position and $u(t)$ a control command (an illustrative proportional law).

Open-loop control executes commands without using output feedback to correct that operation. Murphy also uses **ballistic control** for a trajectory computed and then executed without in-flight correction.

The two distinctions concern different things. CWA concerns knowledge and the completeness of a model; feedback concerns how execution is corrected. An automated manipulator can use closed-loop joint control in a workspace modeled under CWA. It does not thereby acquire open-world task autonomy.

3.6 Greenfield and brownfield automation

Greenfield automation implements automation in an entirely new, undeveloped setting, such as a new factory: layouts and infrastructure can be optimized for the latest robotics and AI technologies without the constraints of existing systems. **Brownfield automation** applies automation to existing facilities with established infrastructure and processes: it often requires retrofitting or adapting existing systems, balancing the benefit of reusing current assets against integration complexity.

This distinction concerns deployment conditions. It does not classify a robot as autonomous or automated. Both settings can contain a combination of automation and autonomous capabilities.

3.7 A toy planning problem and its hidden assumptions

In the **monkey and banana problem**, a simplified agent must move a box under a hanging banana, climb it and obtain the banana. A symbolic solution works because the problem description supplies suitable objects and actions.

S04 uses it to show that the *real world is not a closed world*: Pat Hayes presented the paradox at AISB-1976 in Brighton (the first conference of the British AI association and first European AI conference). The real world adds questions: can the box support the agent? Is it movable? Does climbing succeed? Are there obstacles? A planner can produce a valid solution for its formal model while that solution fails physically because the model omits a relevant condition.

3.7.1 STRIPS: a compact illustration

Murphy introduces **STRIPS** to explain symbolic planning. Represent a state S as a set of facts. An operator specifies **preconditions**, an **add list** and a **delete list**. If the preconditions hold, its modeled successor state is

$$S' = (S \setminus \text{Delete}(a)) \cup \text{Add}(a).$$

For an illustrative **climb-box** operator:

- Preconditions: the agent is next to the box, is on the floor, and the box is climbable.
- Delete list: the agent is on the floor.
- Add list: the agent is on the box.

If **box-is-climbable** is omitted from the model, the planned climb can fail. The formula describes the expected symbolic effect, not evidence that the action succeeded physically. STRIPS (developed for Shakey at SRI) is a planning method, not an entire robot architecture.

3.7.2 Means-ends analysis and a failed precondition

In Murphy's presentation, STRIPS uses **means-ends analysis (MEA)**: compare the current state with the **goal state**, use a **difference evaluator** to identify what is missing, and consult a **difference table** of operators and effects. A failed precondition becomes a **subgoal**; recursion solves that subgoal before returning to the original goal. Planning updates a copy of the world model before the physical execution.

A STRIPS implementation therefore needs three things from the designer: a **world model** representation (predicates such as **INROOM**(x, r), **NEXTTO**(x, t), **STATUS**(d, s), **CONNECTS**(d, rx, ry)); a **difference table** listing operators with preconditions, add list and delete list; and a **difference evaluator** (here, logical subtraction of the current state from the goal state).

Two-room example (Murphy pp. 46–52). Robot IT is in R1; door D1 connects R1 and R2 and is open; box B1 is in R2. Goal: **INROOM**(IT, R2).

Operator	Preconditions	Add list	Delete list
OP1 GOTODOOR(IT, dx)	INROOM(IT, rk), CONNECTS(dx, rk, rm)	NEXTTO(IT, dx)	—
OP2 GOTHRUDOOR(IT, dx)	CONNECTS(dx, rk, rm), NEXTTO(IT, dx), STATUS(dx, OPEN), INROOM(IT, rk)	INROOM(IT, rm)	INROOM(IT, rk)

1. *Initial state*: **INROOM**(IT, R1), **INROOM**(B1, R2), **CONNECTS**(D1, R1, R2), **CONNECTS**(D1, R2, R1), **STATUS**(D1, OPEN). Difference with the goal: $\neg \text{INROOM}(\text{IT}, \text{R2})$.

2. Search the *add lists*: OP2 with $rm=R2$ produces the goal. Binding preconditions against the world model gives $dx=D1$, $rk=R1$. $NEXTT0(IT,D1)$ is false: a **failed precondition**. Push goal G0 on a stack and make $NEXTT0(IT,D1)$ the subgoal G1 (recursion).
3. OP1 adds $NEXTT0(IT,D1)$; its preconditions hold. Push OP1 on the plan stack and apply it to a *copy* of the world model. *State after OP1*: initial state + $NEXTT0(IT,D1)$.
4. Pop back to OP2: all preconditions now hold. Apply it. *State after OP2*: $INROOM(IT,R2)$ replaces $INROOM(IT,R1)$; $NEXTT0(IT,D1)$, B1, the doors and status are unchanged.
5. No difference remains. The plan to execute is $GOTODOOR(IT,D1)$, then $GOTHRUDOOR(IT,D1)$.

All of this is a “mental operation”: STRIPS plans, it does not execute while planning. Naming an action that achieves the goal is insufficient; its preconditions must also be achieved.

3.8 The frame problem and world-model maintenance

The **frame problem** concerns representing the effects of an action while accounting for what remains unchanged, without explicitly enumerating every unaffected fact. When the agent climbs a box, its support relation changes; many other facts, such as another object’s location, should persist.

Murphy’s discussion on printed p. 53 uses the term more broadly to emphasize the difficulty of maintaining a realistic, computationally tractable world model. Both readings expose the same design pressure: adding more facts and possible changes can make reasoning and maintenance unwieldy.

CWA and the frame problem are related but distinct. CWA assumes that the relevant model is complete. The frame problem concerns how state changes and persistence are represented and maintained. Simply adding more details does not necessarily solve either issue efficiently.

3.9 Choosing how much autonomy to use

1. Identify the task and the conditions that can be controlled or modeled.
2. Identify changes that could invalidate the prepared plan.
3. Decide which changes the robot can sense and respond to locally.
4. Allocate uncertain or difficult decisions to a capable robot component or a human supervisor.
5. Define when to revise the plan, request intervention or stop the operation.

Three cases. Repeated welding with fixed fixtures favors automation. Delivery through changing corridors benefits from autonomous perception and route adaptation. An unfamiliar remote manipulation task may benefit from teleoperation with local assistance. The useful choice follows the task, rather than maximizing autonomy for its own sake.

4 Organizing intelligence: robot paradigms and architectures

Reading anchors: S04, PDF pp. 19–30; S02, PDF pp. 24–32; Murphy, printed pp. 5–12, 42–43, 54–63 and 83–91.

4.1 Paradigm versus architecture

A **paradigm** is a philosophy or set of assumptions and/or techniques characterizing an approach to a class of problems (Murphy’s analogy: Cartesian versus polar coordinates for the same calculus problem). Robotic paradigms are described in two ways: (1) by the *relationship among SENSE, PLAN and ACT*, using the input/output table of Section 1, and (2) by the *sensing organization*, i.e. whether sensor data is processed locally for each function or first fused into one global world model. An **architecture** provides a principled way of organizing a control system and describes a set of components and how they interact; it also imposes constraints on how the problem is solved. The same paradigm can have several architectures, as one engine type has many car models.

Paradigm	Period (Murphy)	Why it appeared
Hierarchical	1967–1990 (from Shakey)	Introspective view of human thinking: see, decide, plot a course.
Reactive	heavily used 1988–1992	Interest in biology and cognitive psychology; cheap, more powerful hardware (insect-like robots for under \$500 versus the very costly Shakey).
Hybrid	1990s to today	Throwing out planning was too extreme for general-purpose robots; keep reactive speed and add deliberation.

The two loops of S04 (pp. 20–27). The slides organize intelligence with a brain analogy:

- **Loop 1, deliberative** (“upper brain” or cortex): thoughtful, conscious reasoning over symbols about goals — planning, optimization, situation awareness. This is the *hierarchical architecture*.
- **Loop 2, reflexive/reactive** (spinal cord and “lower brain”): skills and responses, unconscious; “smart as a cockroach, fish, bird, armadillo”. This is the *reactive architecture*.
- A “middle brain” converts sensor data into symbols. Together these give the **deliberative layer**, the **reactive (or behavioural) layer** and the **interfacing layer** between them.

The analogy organizes functions; it does not claim the software reproduces neuroanatomy.

4.2 The Hierarchical Paradigm

The **Hierarchical Paradigm** follows

$$\text{SENSE} \longrightarrow \text{PLAN} \longrightarrow \text{ACT}.$$

Sensing builds or updates a **world model**; planning uses that model to select actions; execution carries out the resulting directives. In the classical organization, sensing is **monolithic**: observations are fused into a shared global representation.

The world model can contain an a priori map, current observations, an estimate of the robot’s state and task knowledge. It is broader than a geometric map. For delivery, it may also encode the room receiving a package and the conditions required for delivery.

Slide example: guided pick (S04 p. 22). A camera observes a part (SENSE), the system computes the grasp (PLAN), then the manipulator picks it (ACT). It works because the workcell is structured: nothing changes between looking and picking.

Advantages and disadvantages (Murphy pp. 61–62).

- *Advantage:* it provides an ordering of the relationship between sensing, planning and acting; explicit representations support goal reasoning and explaining why a plan works. Murphy also notes that hierarchical architectures support a gradual evolution from semi-autonomous to fully autonomous control.
- *Disadvantage — planning bottleneck:* every update cycle the robot updates a global world model and then plans; with slow algorithms this is a bottleneck, and the world can change meanwhile.
- *Disadvantage — sensing and acting are always disconnected:* there are no stimulus–response actions (“a rock is falling on me, move anywhere”).
- *Disadvantage — global world model:* it leads to the frame problem and requires a closed world assumption; STRIPS reasons over irrelevant details (other rooms, other doors) just to open one door.
- *Disadvantage — uncertainty* is not really handled (see below).

4.2.1 NHC and NIST RCS: what the components do

Murphy’s **Nested Hierarchical Controller (NHC)** separates planning into a **Mission Planner** (translate a mission into a goal), **Navigator** (select a path and waypoints), and **Pilot** (select actions for the current path segment). A low-level controller converts directives into actuator commands. All use the relevant world-model information.

NHC interleaves planning and execution: the Pilot monitors progress; reaching a waypoint advances the path, while an obstruction returns control to the Navigator for replanning. Every sensor update need not restart mission planning. Its decomposition is particularly natural for navigation, and less directly applicable to manipulation.

Murphy’s account of **NIST Real-time Control System (RCS)** includes **Sensory Perception** for preprocessing, **World Modeling** with a **Knowledge Database**, **Value Judgment** for evaluating plans, and **Behavior Generation** for executable commands. **NASREM** adapts this organization to space teleoperation. These descriptions concern the historical architectures in the supplied edition.

Evaluation with the four criteria (Murphy pp. 59–61).

- *Modularity:* both give intuitive guidelines for decomposing the program, but the decomposition is strictly functional and gives little guidance for reusable components (monolithic rather than object-oriented programming).
- *Niche targetability:* reasonable — NHC is focused strictly on navigation; RCS is broader and was used for mining equipment, submarines, cars and floor cleaning.
- *Portability:* unclear; the architectures are expressed at a very broad level (“a house should have bedrooms, bathrooms and a kitchen”), and code for a mining machine is hard to reuse for a submarine.
- *Robustness:* RCS’s Value Judgment simulates a plan before execution. This is a limited form of robustness, feasible in well-known environments such as a nuclear processing cell, and it adds the delay of “mental rehearsal” (a falling ceiling requires an immediate move).

Murphy’s summary: hierarchical systems fell out of favor (except NIST RCS) because they favored strong niche targetability at the expense of true modularity and portability.

Hierarchical execution must also handle **sensor noise**, **actuator errors**, semantic uncertainty (what counts as “next to”?) and **action completion**: issuing a grasp command does not establish that the object was grasped.

4.3 The Reactive Paradigm

The **Reactive Paradigm** directly couples sensing and acting:

$$\text{SENSE} \rightarrow \text{ACT}.$$

A **behavior** is a task-relevant perception–action coupling, such as avoiding obstacles or moving toward a visible goal. Multiple behaviors can run concurrently. Each uses the information needed for its function, rather than waiting for a complete global description of the scene.

Slide example: a vacuum robot at the stairs (S04 p. 25). Sensor A looks down at the floor edge, sensors B and C look ahead. In the reactive organization each sensor is coupled directly to a response (no floor below: stop or turn away from the stairs; obstacle ahead: turn). The slide shows three parallel SENSE–ACT loops, with no map and no planner.

Example. A move-toward-goal behavior proposes forward motion; an avoid-obstacle behavior proposes a turn. The final action depends on a coordination mechanism, such as arbitration or an appropriate combination. Without coordination, two behaviors can issue incompatible commands.

Emergent behavior describes a system-level result produced by interactions among simpler behaviors. A curved path around an obstacle may emerge even when neither behavior individually specifies that exact path. Emergence is not a guarantee of success: the coordination mechanism must still suit the task.

Strength. Short perception–action paths can respond quickly and reduce dependence on a complete world model.

Limitation. Purely local responses may be insufficient for tasks requiring memory, long-term ordering or a route that initially moves away from a goal. Eliminating explicit planning does not eliminate every difficult decision.

Key idea. Reactive does not mean random or sensorless. It means that action is closely coupled to task-relevant perception, without requiring a deliberative planning stage for each response.

4.4 Perception for action and perception for recognition

The distinction between two perceptual functions explains why the same robot may need both reactive and deliberative processing.

Direct perception extracts action-relevant information without first constructing a complete symbolic interpretation. A robot may detect that something is approaching and respond before deciding precisely what object it is.

An **affordance** is a perceivable possibility for action offered by the environment relative to an agent’s capabilities. An opening affords passage only if the robot can fit through it. A surface may afford support only for an agent of suitable size and weight. Affordances link perception to what the agent can do.

Recognition identifies objects or specific instances using models, memory or interpretation. Avoiding a nearby object may require only its position; delivering *this user’s cup* requires distinguishing it from other cups. The correct processing depends on what the task needs.

Murphy uses Gibson’s ecological approach for affordances and Neisser’s distinction between direct perception and recognition. Direct perception is an organizing ideal; its robotic implementation can

still require substantial signal or image processing. The absence of symbolic recognition does not make extracting the needed information free.

4.5 The Hybrid Deliberative/Reactive Paradigm

The **Hybrid Deliberative/Reactive Paradigm** combines goal reasoning with fast behaviors. Its characteristic organization is

PLAN, SENSE–ACT.

The comma matters: the robot first plans (deliberates) how to decompose a task into subtasks (**mission planning**) and which behaviors suit each subtask; then the behaviors execute as in the Reactive Paradigm. S04 p. 30: “PLAN, then instantiate appropriate SENSE-ACT behaviors, until next step in plan”; PLAN requires a world model, though a bounded one, plus the planning algorithms. The S04 hybrid slide (p. 29) draws PLAN connected to both SENSE and ACT and shows the same robot with a “new approach”: inputs from sensors A, B and C are processed together on one board before the output to the motor, instead of three independent couplings. Planning is revisited when a subtask completes, the situation changes or execution fails. The reactive layer need not wait for a full new mission plan before responding to an immediate condition.

Component	Main responsibility	Typical information
Deliberative layer	Select goals, decompose tasks and plan.	World model, history, task constraints.
Interfacing layer	Connect plans to executable behaviors and return status.	Behavior parameters, progress, failure reports.
Reactive / behavioral layer	Execute skills and respond to current conditions.	Current task-relevant sensor information.

The two decision loops operate on different **time horizons**. Reactive control focuses on the immediate present. Deliberation may use the past, predict future consequences and compare alternatives. Sensing organization is mixed: sensor data is routed to each behavior that needs it, but is also available to the planner for a task-oriented global world model; the planner may “eavesdrop” on the behaviors (an obstacle found by a behavior is put into the map). Each function runs at its own rate: planning may update every few seconds (Murphy’s example: 5 s), behaviors many times per second (1/60 s).

Example: corridor blocked during delivery.

1. The deliberative layer selects a route and activates corridor-following behaviors.
2. The reactive layer responds immediately to a detected obstruction.
3. The interface reports that the route cannot be executed as expected.
4. The world model is updated and the planner chooses another route or requests assistance.

A local response buys time; deliberation resolves the broader task consequence.

4.6 Why the interface is difficult

A measurement and a symbolic fact are different representations. A range observation may support “obstacle ahead,” but it may not establish “door permanently blocked.” The interface must communicate appropriate information about observations, task progress and failures.

The world model must contain enough information for useful decisions while remaining tractable. Maintaining all details at all times is expensive and can slow the very decisions needed for adaptation. Conversely, discarding all history makes some tasks impossible.

S04 p. 28 (“the hard part”) lists what changes from reactive to deliberative: two types of perception, **direct** and **recognition** (symbols), which impacts computer vision; different time horizons, which impacts sensing, storage and the algorithms for reasoning and projecting; and the need for a central, tractable **world model** holding symbols, history and knowledge.

Interaction adds another requirement, a **theory of mind**: the robot may need to represent another agent’s **beliefs, desires and intentions (BDI)** and establish **common ground**: a shared understanding of the task and what a command means. “Bring the cup” needs agreement about which cup and where it should go. These are additional representation problems, not automatic consequences of having a planner.

4.7 The three paradigms compared

Feature	Hierarchical	Reactive	Hybrid
Organization	Sense–Plan–Act.	Concurrent Sense–Act behaviors.	Plan plus Sense–Act execution.
Sensing	Shared global model.	Local, behavior-specific information.	Both local inputs and a bounded shared model.
Main benefit	Explicit goal reasoning.	Fast responses.	Long-term goals with immediate adaptation.
Main challenge	Model accuracy and planning latency.	Coordination and limited long-term reasoning.	Layer interfaces and consistent execution status.

Key idea. A hybrid architecture combines complementary capabilities. It still needs suitable models, behaviors and interfaces; the word “hybrid” alone does not establish robustness.

4.8 Evaluating an architecture

Murphy’s four criteria (adapted from Arkin) provide a practical vocabulary for comparing implementations; they are applied to NHC and RCS above:

1. **Support for modularity:** are responsibilities and interfaces clear enough to maintain and reuse components?
2. **Niche targetability:** does the architecture fit its intended task and environment?
3. **Ease of portability to other domains:** how readily can it transfer to different tasks or robot platforms?
4. **Robustness:** where is it vulnerable, and how are those vulnerabilities addressed?

Niche targetability and portability can conflict: a specialized solution may fit one task well while being difficult to reuse. Likewise, dividing a diagram into boxes is not enough to guarantee reusable code. Apply the criteria to concrete components and failure cases.

5 Book supplement: biological foundations of behavior

Reading anchors: Murphy, Chapter 3, printed pp. 67–100. Book material: S04 introduces the reactive architecture, Murphy Ch. 3 explains its biological foundations.

5.1 Biological inspiration and computational theory

Biology supplies examples of agents succeeding with tightly coupled perception and action. The engineering objective is to extract useful principles, rather than reproduce every biological structure. Murphy distinguishes three levels of a **computational theory**:

1. **What can or should be done:** establish a function, such as approaching a target.
2. **Inputs, outputs and transformations:** describe the process, such as mapping a detected target direction to a steering command.
3. **Implementation:** choose the mechanism or algorithm that performs that transformation.

A robot and an animal may share a function and process while implementing them differently. **Braitenberg vehicles** illustrate how simple sensor–motor couplings can produce apparently purposeful motion.

5.2 Behavior categories and acquisition

A **behavior** maps sensory inputs to a pattern of motor actions supporting a task. Murphy’s ethological categories are **reflexive** (innate stimulus–response), **reactive** (a learned skill executed without conscious attention) and **conscious** (deliberative). The word *reactive* in robotics is used more broadly for close perception–action coupling; it does not imply that the behavior was learned.

The reflexive categories are:

- **Reflexes:** the response persists while the stimulus persists and scales with it.
- **Taxes** (singular **taxis**): orientation or movement relative to a stimulus, such as approaching light or moving away from it.
- **Fixed-action patterns:** a response continues beyond the initiating stimulus, such as fleeing after the threat disappears from view.

Behaviors can be innate, a sequence of innate behaviors, innate with memory or initialization, or learned. Innateness and memorylessness are different properties. An agent may have an innate navigation routine while learning its home location.

5.3 Innate Releasing Mechanisms and coordination

An **Innate Releasing Mechanism (IRM)** specifies when a behavior becomes active. A **releaser** is an activation condition, possibly combining an external observation with **internal state** or **motivation**. Low battery and a visible charging station can jointly release a charging behavior.

Illustrative control logic:

```
if immediate_threat:  execute escape_behavior
else if battery_low and station_visible:  approach_station
else:  execute current_task
```

The priority structure prevents simultaneous execution of incompatible commands. IRMs model intelligence at *Level 2* of a computational theory: they describe the process, not the implementation. Their weakness is that interactions between concurrent behaviors are not easy to identify or diagram.

Implicit chaining lets completion of one behavior change the internal or environmental conditions that release the next. An explicit sequence that runs each behavior to completion can delay responses to a new stimulus. The timing and activation logic are therefore part of the design.

Concurrent behaviors can yield **equilibrium**, **dominance** (winner takes all), or **cancellation**. **Inhibition** disables a competing behavior. Coordination must be selected deliberately; simply running every released behavior does not ensure a useful action.

5.4 Action–perception cycle and two uses of perception

An action changes the world or the agent’s viewpoint, affecting its next observation. That observation guides another action: the **action–perception cycle**. This cycle does not require explicit planning at every update.

Perception has two distinct roles: **releasing** a behavior and **guiding** it after activation. The same percept can do both: a fish responds to a lure (release) and follows it (guide). **Action-oriented perception** extracts information relevant to that behavior. **Selective attention** determines which information to examine next.

Active perception changes the agent’s action to gather better information before continuing the primary task, for example moving to see around an occluder. This term describes a perception strategy; it does not by itself classify the sensor hardware as active or passive. The hardware classification requires later sensor material.

Gibson’s **affordances** and Neisser’s **direct perception versus recognition** are explained in the architecture section. Murphy’s **optic flow** example uses apparent image motion to support action, including **time to contact**; it illustrates action-relevant perception rather than semantic recognition of every object.

5.5 Schema theory: connecting perception and motion

A **schema** is a reusable template of data and computation. A **schema instantiation (SI)** supplies concrete parameters, analogous to constructing an object from a class.

A primitive behavior combines a **perceptual schema** for extracting a percept and a **motor schema** for converting it into action. Instantiating `move_to_target` supplies a specific target. A **gain** can scale the motor response.

A **meta-behavior** can sequence primitives or choose alternative perceptual/motor schemas, such as switching perception when vision is unsuitable.

In Murphy’s **rana computatrix**, two targets produce two motor vectors; their sum points between the targets. This demonstrates emergence and potentially unintended composition. Schema theory does not prescribe a universal coordination rule.

Key idea. IRM answers “when is the behavior active?” A perceptual schema answers “what information does it use?” A motor schema answers “how does that information become action?” Coordination answers “what happens if multiple behaviors are active?”

5.6 Transfer to robots: what remains difficult

Murphy’s principles extracted from natural intelligence (p. 100):

- Decompose complex actions into independent behaviors that tightly couple sensing and acting; behaviors are inherently parallel and distributed.
- Use straightforward, boolean activation mechanisms (e.g. IRMs) to simplify control and coordination.
- Filter sensing to what is relevant to the behavior (action-oriented perception).
- Direct perception (affordances) reduces the computational complexity of sensing and allows action without memory, inference or interpretation.
- Behavior outputs may be combined into a resultant output or one may inhibit another (competing-cooperating).

Open issues include resolving conflicts, deciding when memory or symbolic knowledge is necessary, and acquiring new behavior sequences. Emergent behavior cannot be completely predicted, which makes end users such as the military, NASA or the nuclear industry reluctant to accept robots without guarantees. A white-line follower distracted by a judge's white shoes illustrates an unintended stimulus.

6 Revision: explain, compare, apply

6.1 The essential chain of reasoning

1. A robot acts physically through actuators and obtains information through sensors.
2. Engineered, repetitive tasks can often use a prepared plan and a sufficiently complete model.
3. Unknowns and changes require current sensing, execution monitoring and adaptation.
4. Hierarchical organization supports explicit plans; reactive organization supports immediate responses.
5. Hybrid organization links these capabilities, while human-robot interaction adds control allocation and shared understanding.

6.2 Questions with concise model answers

1. Is every robot an intelligent robot?

No. A robot may mainly replay a prepared sequence. An intelligent robot uses information about its situation to select or adapt actions toward a task goal.

2. Why does field robotics make perception important?

Its environment cannot usually be arranged and maintained like a production fixture. The robot must observe relevant changes instead of relying only on a prepared description.

3. What separates telepresence from autonomy?

Telepresence improves the human's experience of the remote environment. Autonomy concerns which decisions and tasks the robot can perform independently.

4. How do shared control and control trading differ in Murphy?

Shared control involves assistance or monitored delegation with human involvement. Control trading delegates a task the robot can complete autonomously until a new command or interruption.

5. Explain automation versus autonomy without using robot shape.

Automation emphasizes repeated execution under well-modeled conditions. Autonomy emphasizes adaptation when relevant conditions or outcomes are uncertain. Both can exist in the same robot.

6. What happens to an unlisted obstacle under CWA and OWA?

Under CWA, the model treats it as absent. Under OWA, its absence from the model leaves its existence unknown. Current observations are needed to determine whether a route is clear.

7. Can closed-loop control be used under CWA?

Yes. Feedback may correct a joint's position while the workspace and task are assumed complete and known. Model completeness and feedback are different properties.

8. Why can a valid symbolic plan fail?

It is valid relative to its model. An omitted condition, an unexpected event or an action whose physical effect differs from its modeled effect can prevent execution.

9. What is the frame problem?

Representing the effects of actions and the persistence of unaffected facts without an impractical enumeration. Murphy also emphasizes the broader burden of maintaining a tractable world model.

10. Is STRIPS a robot architecture?

No. It is a symbolic planning method. An architecture must also organize perception, execution and the information exchanged among components.

11. What separates hierarchical and reactive sensing?

Hierarchical sensing is fused into a shared world model for planning. Reactive sensing is distributed according to the needs of individual perception–action behaviors.

12. What does an affordance add to object recognition?

It describes an opportunity for action relative to the agent. Detecting a passage wide enough for a robot can be more useful for navigation than naming every object around it.

13. Why combine deliberative and reactive layers?

Deliberation supports goals, memory and future consequences; reactive behaviors support rapid responses to current conditions. An interface makes their decisions and status useful to each other.

14. Is collaborative robotics the same as a hybrid architecture?

No. Collaboration concerns human–robot work. A hybrid architecture concerns the robot's internal organization of planning and behavior execution.

15. What are the three levels of a computational theory?

Define the function, decompose it into inputs/outputs and transformations, then choose an implementation. Biological and robotic agents can share the first two without sharing the third.

16. What is an IRM, and what can be a releaser?

An Innate Releasing Mechanism specifies activation of a behavior. A releaser can be an external stimulus, an internal state, or a combination, such as low battery and a visible station.

17. How do perceptual and motor schemas compose a behavior?

The perceptual schema extracts the relevant information; the motor schema maps it into action. Instantiation supplies parameters for a concrete situation. The releaser determines activation.

18. How does STRIPS handle a failed precondition?

In Murphy's means–ends presentation, the failed precondition becomes a subgoal. The planner recursively finds operators to achieve it, updates a modeled state, and returns to the original goal before physical execution.

19. Name the four criteria for evaluating an architecture.

Support for modularity, niche targetability, ease of portability to other domains, and robustness. Explain each with respect to the intended application and its failure cases.

20. Does NHC repeat all planning after each observation?

No. The Pilot monitors the current path segment; a reached waypoint advances navigation, while an obstruction can return control to the Navigator. The entire mission need not be recomputed.

21. Is active perception the same as an active sensor?

No. Active perception changes an action to obtain better information. Whether the sensor hardware is active or passive is a separate classification (later sensing lessons).

22. Define SENSE, PLAN and ACT by their inputs and outputs.

SENSE: sensor data $\rightarrow$ sensed information. PLAN: sensed and/or cognitive information $\rightarrow$ directives. ACT: sensed information or directives $\rightarrow$ actuator commands.

23. Describe the three paradigms by primitives and sensing organization.

Hierarchical: S, P, A with sensing fused into a global world model. Reactive: concurrent S–A behaviors with local, behavior-specific sensing. Hybrid: P, S–A; sensing routed to behaviors and also to the planner’s task-oriented world model.

24. Classical AI paradigm versus intelligent robotics paradigm?

Classical AI reasons over an abstract, fully specified problem (chess) in a one-way perceive–reason–act chain. Intelligent robotics closes the loop through the real, partially known environment; autonomous behavior emerges from continuous perception–action interaction.

25. Why does robotics say “self-governing” without meaning moral independence?

The term comes from the mechanical tradition (Watt’s steam-engine governor): a machine regulating itself toward a goal. The dictionary sense of moral independence feeds the myth of evil robots; bounded rationality further limits any agent.

26. Give the four automation/autonomy keyword pairs of S04.

Plans: execution vs generation. Actions: deterministic vs non-deterministic. Models: closed world vs open world. Knowledge representation: signals vs symbols. Automation is about tools, autonomy about agents.

27. Give two advantages and two disadvantages of the Hierarchical Paradigm.

Advantages: a clear ordering of sensing, planning and acting; explicit goal reasoning (and a path from semi-autonomous to autonomous control). Disadvantages: the planning bottleneck on a global world model (frame problem, CWA); sensing and acting always disconnected, so no stimulus–response; uncertainty poorly handled.

28. Evaluate NHC/RCS with the four criteria.

Modularity: intuitive but only functional decomposition. Niche targetability: reasonable (navigation; RCS on mining, submarines, cars). Portability: unclear, specified too broadly. Robustness: limited, via RCS plan simulation, at the price of delay.

6.3 One scenario across the whole course

Task: a robot brings a component to a worker, but someone leaves a trolley in the corridor.

Automation perspective. If the design assumes a fixed clear route, replaying the prepared sequence may fail. A low-level feedback controller can track the requested motion without understanding that the route itself is unsuitable.

Open-world perspective. The prior map does not prove present clearance. The robot must detect the trolley and update its understanding of the route.

Reactive response. A local behavior slows or stops the robot in response to the obstruction. This response alone may leave the delivery unfinished.

Deliberative response. A planner evaluates an alternative route using the updated model and the delivery goal.

Hybrid response. The local behavior handles the immediate obstruction while the planning layer resolves the route. The interface connects the observed failure to the task plan.

Shared-intelligence response. If the route cannot be resolved, the robot can report the problem to a human. The human may change the goal or give contextual information while the robot

performs the local execution.

6.4 Common mistakes to avoid

- Using *closed world* and *closed-loop control* interchangeably.
- Treating an unknown fact as false in an open-world task.
- Equating AI with machine learning, or intelligence with humanoid appearance.
- Assuming autonomy means the complete absence of a human supervisor.
- Assuming reactive behaviors do not need coordination or computation.
- Assuming a world model is only a geometric map.
- Using *collaborative*, *shared control* and *hybrid architecture* as synonyms.

7 Source map and editorial scope

7.1 Source identifiers

- S02** Emanuele Menegatti, *Introduction to Intelligent Robotics 26–27*, supplied slide set 02, 32 PDF pages.
- S03** Emanuele Menegatti, *The (R)Evolution of Robotics*, supplied slide set 03, 27 PDF pages. The cover is labeled A.Y. 2024–25.
- S04** Emanuele Menegatti, *Robot Automation vs Autonomy and Closed World vs Open World Assumption*, supplied slide set 04, 30 PDF pages. The cover is labeled A.Y. 2024–25.

Murphy

Robin R. Murphy, *Introduction to AI Robotics*, MIT Press, 2000. The supplied extract contains the first 124 PDF pages, ending on printed p. 103.

Page convention. Slide references use the one-based PDF page index, because printed slide numbers are not consistently aligned. Book references use printed page numbers. In this extract, a numbered book page p is on PDF page $p + 21$; printed p. 53 is PDF page 74.

7.2 Which material supports each part?

Notes	Primary slides	Book integration
Foundations	S02 pp. 2–32.	Part I introduction, pp. 2–10 (Fig. I.2); Ch. 1, pp. 13–16, 21–24 and 28–37.
Evolution and collaboration	S03 pp. 2–27; S02 p. 32.	Ch. 1, pp. 13–28. Industry 4.0/5.0, cobot and humanoid data come from the slides.
Automation and autonomy	S04 pp. 2–18.	Ch. 1, pp. 13–16; Ch. 2, pp. 42–53 (STRIPS table, pp. 47–52). Greenfield/brownfield and the Watt governor come from S04.
Architectures	S04 pp. 19–30; S02 pp. 24–32.	Part I, pp. 5–12; Ch. 2, pp. 42–43 and 54–63; Ch. 3, pp. 83–91.
Biological foundations	Book supplement; related to S04 reactive overview.	Ch. 3, pp. 67–101.

7.3 Editorial choices

Repeated diagrams, product galleries and video prompts have been condensed or omitted; only the cobot and humanoid market indications are kept as orders of magnitude. The retained history explains the difference between structured industrial work and adaptation in less predictable environments.

The supplied 2000 edition supports the paradigm overview and the first three chapters. S03 also includes a timeline credited to Murphy’s 2019 second edition; that attribution does not mean the second edition is present in this repository. The detailed Hybrid Paradigm chapter of the 2000 book is outside the supplied extract, so the hybrid discussion here relies on the available Part I overview and S04.

Worked examples, the proportional control formula and the revision answers are explanatory synthesis. They are not presented as quotations from the sources. No later lessons, exam predictions or unavailable textbook chapters have been added.